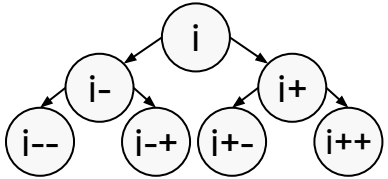
ИТМО

Алгоритмы и структуры данных

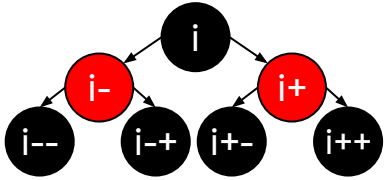
Лекция 7

В-деревья (би-деревья)

Вспомним деревья



Бинарное дерево

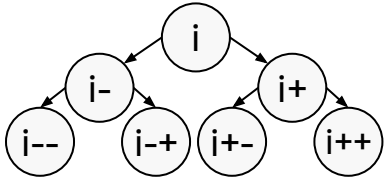


Красно-черное дерево

Вспомним деревья

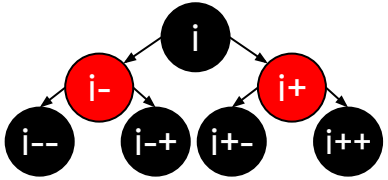
ИТМО

Бинарное дерево



Сложность - O(H), H - высота дерева

Красно-черное дерево



Поиск, вставка, удаление

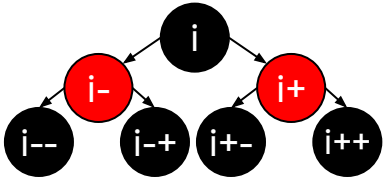
Сложность - O(logN), N - кол-во элементов

Ограничение 1: корень - черный
Ограничение 2: если узел красный -> дети черные
Ограничение 3: путь к листьям содержит одинаковое
число черных узлов

Использование красно-черных деревьев

ИТМО

Красно-черное дерево

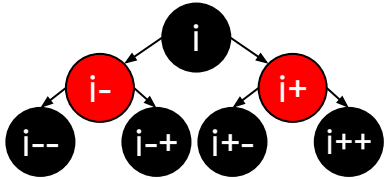


Одно из важных применений: использование этой структуры для индексов в БД

Использование красно-черных деревьев

ИТМО

Красно-черное дерево



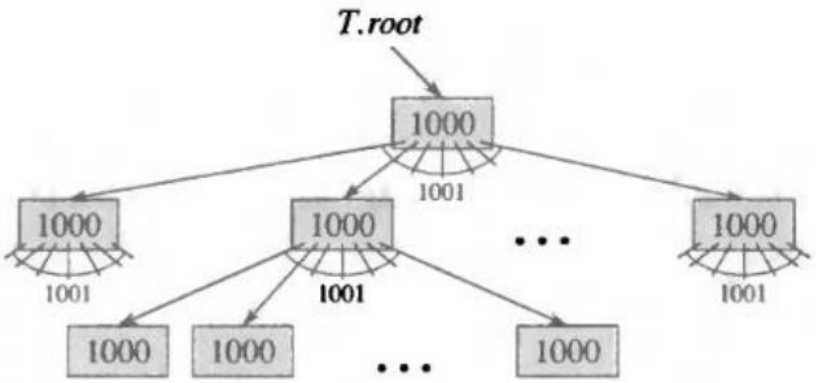
Офф. документация:

11.2. Index Types

11.2.1. B-Tree
11.2.2. Hash
11.2.3. GiST
11.2.4. SP-GiST
11.2.5. GIN
11.2.6. BRIN

B-деревья (btree): по умолчанию (операции равенства и диапазона)
Hash-индексы: поиск по равенству (не поддерживают операции диапазона)
GiST (Generalized Search Tree): Позволяет создавать индексы на основе различных типов данных, включая геометрические и полнотекстовые данные.
GIN (Generalized Inverted Index): для поиска в массиве, JSONB, и полнотекстовом поиске.
SP-GiST: Поддерживает разреженные структуры данных, такие как деревья и графы.
BRIN (Block Range INdex): для очень больших таблиц (упорядоченное распределение данных)

Определение B-деревьев

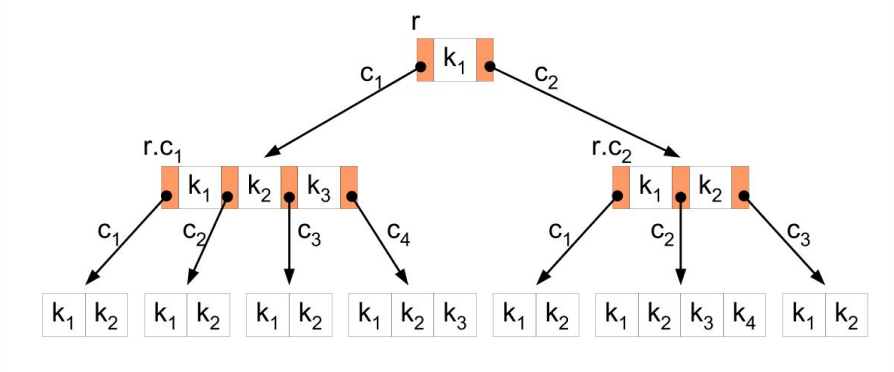


$$h \leq \log_t \frac{n+1}{2}$$

Каждый узел хранит:

1. *x.n* - количество ключей в узле
2. список ключей, при условии *x.k1 < x.k2 < … < x.kn*
3. x.leaf: bool - является ли узел листом
4. x.n + 1 указателей на дочерние узлы
5. Все листья расположены на одной и той же глубине, которая равна высоте дерева *h*.
6. минимальная степень (t ≥ 2):
   a. каждый узел кроме root должен содержать как минимум t-1 ключей
   b. каждый узел содежит не более чем 2t - 1 ключей

Определение B-деревьев



Каждый узел хранит:

1. *x.n* - количество ключей в узле
2. список ключей, при условии *x.k1 < x.k2 < … < x.kn*
3. x.leaf: bool - является ли узел листом
4. x.n + 1 указателей на дочерние узлы
5. Все листья расположены на одной и той же глубине, которая равна высоте дерева *h*.

t – это минимальная степень (minimum degree) B-дерева
t – минимальная степень (t ≥ 2)

Корень содержит от 1 до 2t-1 ключей.
Любой другой узел содержит от t-1 до 2t-1 ключей.

обычно t находится в диапазоне (50; 2000).

Допустим t = 2

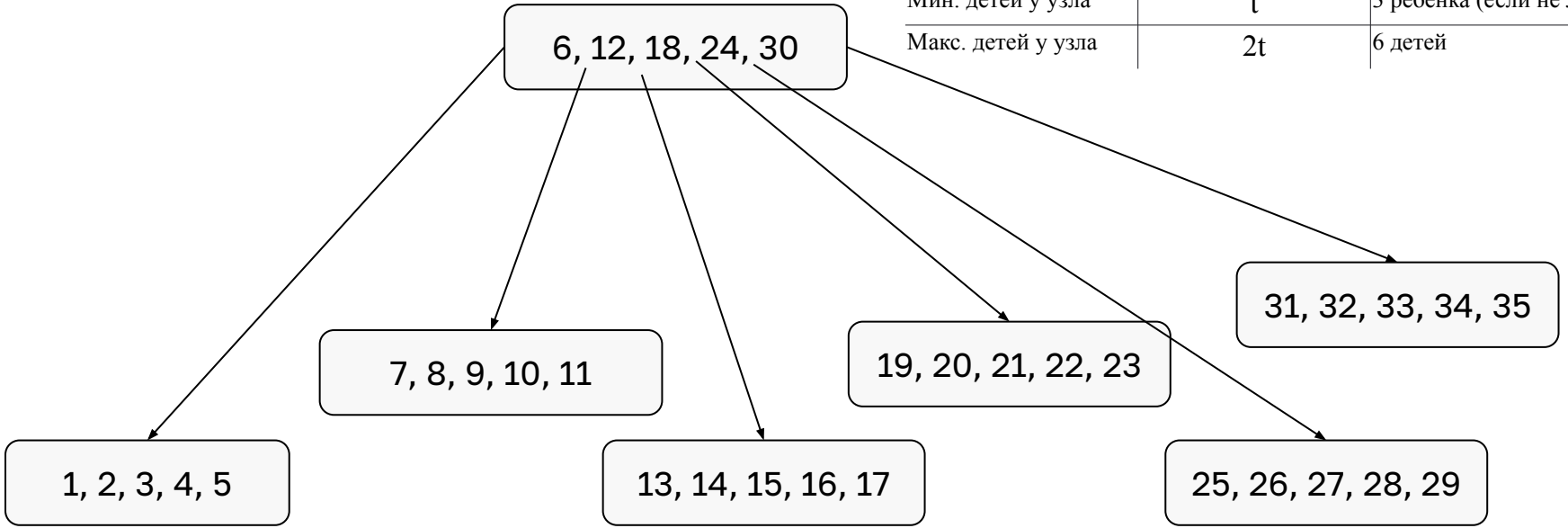
Параметр	**Формула**	**Значение для** $t = 2$
Мин. ключей в узле	$t - 1$	1 ключ (кроме корня)
Макс. ключей в узле	$2t - 1$	3 ключа
Мин. детей у узла	t	2 ребенка (если не лист)
Макс. детей у узла	$2t$	4 ребенка

Допустим t = 3

Параметр	**Формула**	**Значение для** $t = 3$
Мин. ключей в узле	$t - 1$	2 ключа (кроме корня)
Макс. ключей в узле	$2t - 1$	5 ключей
Мин. детей у узла	t	3 ребенка (если не лист)
Макс. детей у узла	$2t$	6 детей

Пример дерева с t = 3

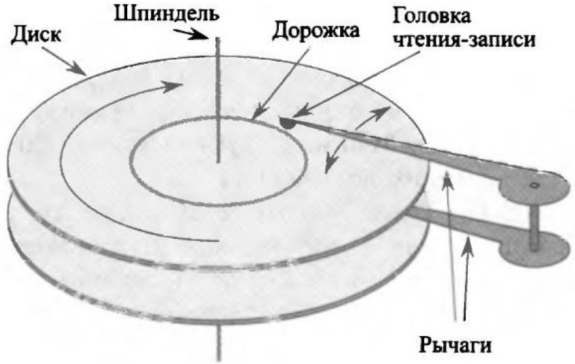
Параметр	Формула	Значение
Мин. ключей в узле	t - 1	2 ключа (кроме корня)
Макс. ключей в узле	2t - 1	5 ключей
Мин. детей у узла	t	3 ребенка (если не лист)
Макс. детей у узла	2t	6 детей



Почему используется B-дерево в БД?

ИТМО

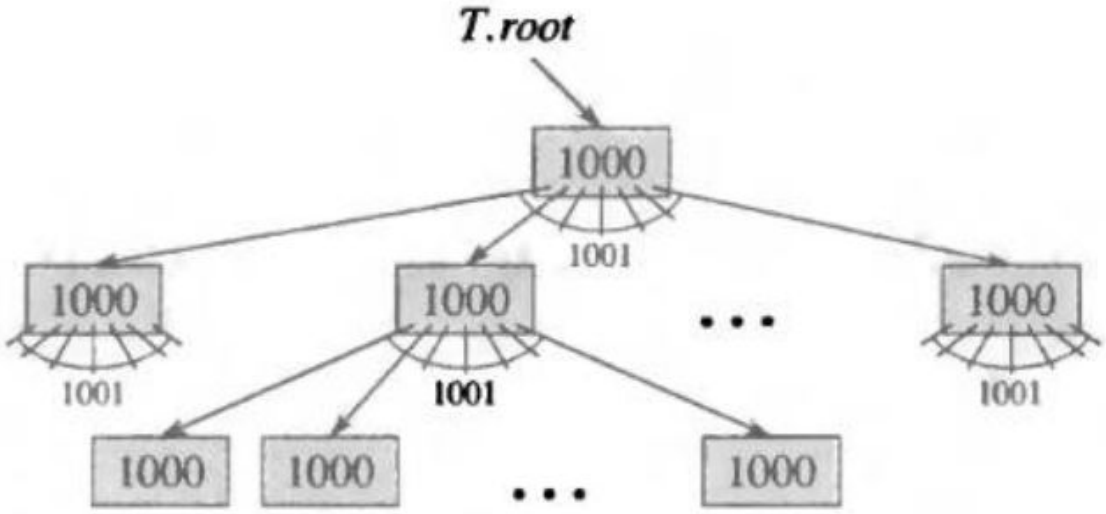
Что есть у Кормена



Тут пообщаемся

Придумать современную задачу для B-деревьев

Возможности B-дерева



1 узел,
1000 ключей

1001 узел,
1 001 000 ключей

1 002 001 узел,
1 002 001 000 ключей

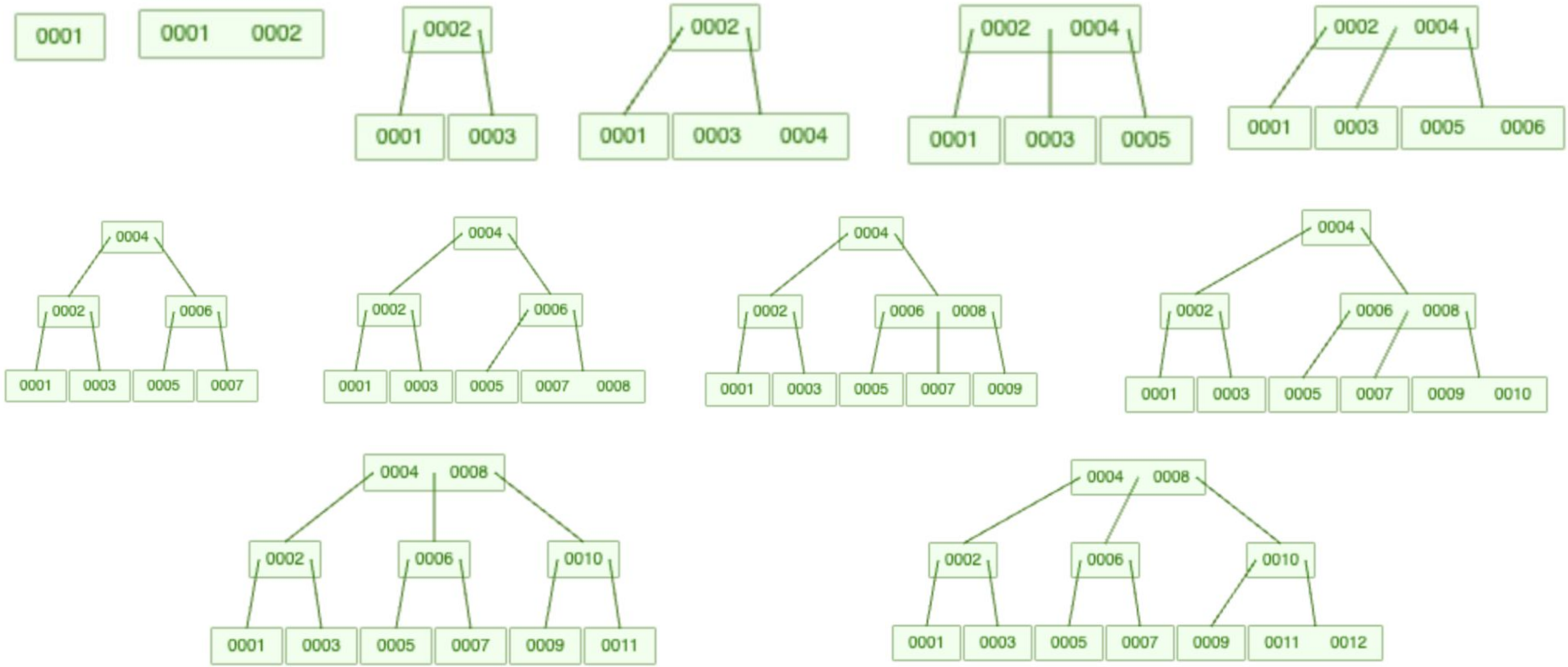
https://habr.com/ru/articles/783012/

Как выглядит вставка в гифке

https://www.cs.usfca.edu/~galles/visualization/BTree.html

Го построим B-дерево (t == 3)

ИТМО

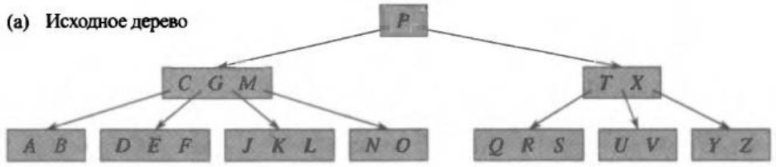


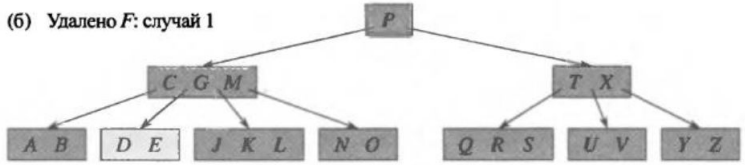
Алгоритм добавления

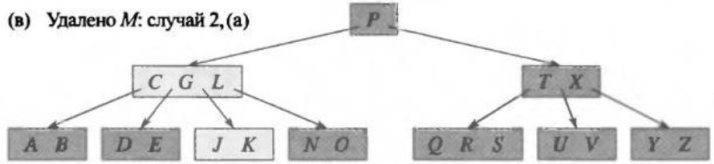
```
INSERT(root, key):
  if root.full(): // Если корень полон - делим его
    newRoot = split(root)
    insertNonFull(newRoot, key)
  else:
    insertNonFull(root, key)  // Если корень не полон - спускаемся в него

insertNonFull(node, key):
  if node.isLeaf():  // МЫ В ЛИСТЕ. Просто вставляем ключ в массив.
    add key to node.keys // Вставляем в нужное место (сохраняя сортировку)
  else:
    i = findChildIndex(node, key)  // Ищем нужного ребенка

    if child[i].full(): // Если ребенок полон - делим
      splitChild(node, i)
      // После сплита ключ может попасть в левый или правый новый узел
      update i if needed

    // 6. Спускаемся в гарантированно НЕ полный узел
    insertNonFull(child[i], key)
```

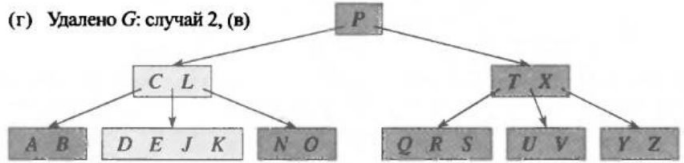
```
splitChild(parent, idx):
  fullNode // Ищем Child
  newNode // Создаем новый

  newNode.keys = fullNode.keys[mid:]
  fullNode.keys = fullNode.keys[:mid]

  if !fullNode.isLeaf():
    // переносим также детей

  parent.addChild(newNode)
  // добавить в parent mid_key
```

Удаление из B-дерева







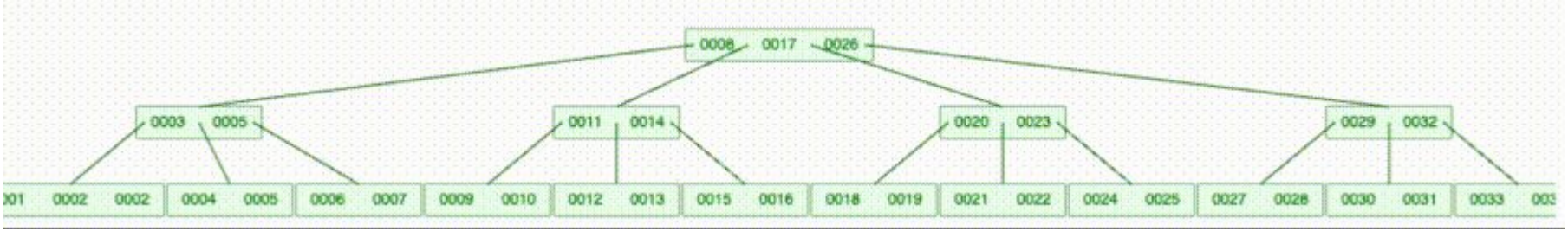


Удаление F. Реализуется случай 1: простое удаление из листа.

Удаление M. Реализуется случай 2: предшественник M (L) перемещается вверх и занимает место M

Удаление G. Реализуется случай 2: G переносится вниз в узел *DEGJK*, а затем выполняется удаление G из листа (случай 1).

Удаление из B-дерева



https://www.cs.usfca.edu/~galles/visualization/BTree.html

Мне кажется

Или у вас есть вопросы?

ITsMOre than a UNIVERSITY

Ivan.Khodnenko@itmo.ru